

Two-stage Pruning During Training for Artificial Neural Networks

Pelinsu Sarac

Abstract—Deep learning has achieved remarkable success in various domains such as Computer Vision and Natural Language Processing. However, the deployment of these high-performance models on edge and mobile devices remains a challenge due to their large number of parameters. Model pruning has emerged as an effective solution to reduce model size while maintaining accuracy. Most of the pruning methods in literature follow the conventional approach of pruning after training, which involves training a model to convergence, removing low-importance weights, and fine-tuning. In contrast, pruning during training has been explored less due to its dynamic nature and additional computational complexity. In this paper, a simple pruning technique is proposed that removes unnecessary weights dynamically during training using a two-stage scoring approach, applied to MLPs and at the filter level for CNNs. In the first stage, weights are evaluated based on their gradient magnitudes, under the assumption that smaller gradients indicate less need for updates, hence proximity to the values in convergence. In the second stage, weights with both low magnitude and small gradients are pruned. This method is periodically applied during training, reducing the number of parameters without significantly affecting accuracy. Experiments on MNIST with MLP and on CIFAR-10 with ResNet18 and VGG16 show that this approach effectively sparsifies the model with minimal accuracy degradation under appropriate hyperparameter settings. Additionally, the results highlight how architectural choices and pruning intervals influence both sparsity and performance.

Index Terms—Deep learning, model pruning, pruning during training, network sparsification, computational efficiency.

I. INTRODUCTION

DEEP learning is one of the topics that has changed the humanity greatly over the past years, with different fields of application such as Computer Vision and Natural Language Processing [1]. Even though the performance achieved by these models are groundbreaking, this performance comes from huge models with billions of parameters and significant computational cost [1]. Because of this need for billions of parameters, these high-performance models cannot be deployed on edge or mobile devices most of the time [2], making it hard for these models to be used directly in real-life applications.

Solving this issue has become a research topic by itself. Different techniques have been developed by researches over the past years [1]. One of the fundamental examples to these techniques was developed by Han et al. [2] as combination of magnitude-based neural network pruning, quantization and Huffman coding. Among the different techniques, pruning has been continued to be explored as it is one of the techniques that has been proven to compress the models effectively without a significant decrease in the performance [1]. In their survey, Cheng et al. [1], has shown that there has been an increase in the papers published on pruning since 2015.

However, an observation that was done by Blalock et al. [3] states that vast majority of these methods suggest slight variations to general outline suggested by Han et al. [2]. This outline includes training the model until convergence and prune the model weights that are the smallest with respect to some scoring system, compared to the layer or filter they are in, or to rest of the model [3]. After the pruning, models are usually fine-tuned to regain the performance pre-pruning [3].

This main outline has been named as *Pruning after training pipeline* by Cheng et al. [1]. Their survey [1] has also identified two other pipelines, namely *Pruning before training* and *Pruning during training*. Among them, *Pruning during training* was said to be explored less due to more complicated dynamic process [1].

The goal of this paper is to come up with a new, simple technique to score the weights while pruning the model during the training. To achieve this goal, a two-stage scoring approach has been developed. In the first stage, weights or filters in a convolutional neural network are selected based on their gradients or mean absolute gradients. Smaller gradients in terms of magnitude have been chosen as an indicator that shows less update is needed for this parameter, thus it is close to its final value after the training. The second stage includes pruning the weights or filters with smaller gradients with respect to their weights or mean absolute weights. In other words, weights with both small gradient and small magnitude are pruned. This process can be repeated every few epochs of the training. This method has been analyzed using different number of epochs as interval for applying the pruning and different percentiles for pruning threshold, on fundamental classification tasks like MNIST [4] and CIFAR-10 [5], using different architectures like MLP [6], ResNet [7] and VGG [8]. The results have shown that the method can indeed make the model sparser without a significant drop in accuracy using the appropriate hyperparameters, and it has also shown that different architectures can be affected differently.

The remainder of this survey is organized as follows. In Section II, previous works that have been done on pruning techniques, especially ones on *Pruning during training* have been presented. In Section III, main motivation behind the technique has been restated. In Section IV, two main algorithms that are implementing the proposed method are presented, along with the experiment design. In Section V, results of the experiments are presented for each architecture-dataset combination. In Section VI, implications of the experiments are discussed. The paper is concluded in Section VII.

The code used for this paper can be found on <https://github.com/psarac/two-stage-pruning-for-ANN/tree/main>.

II. RELATED WORK

The history of pruning for neural networks goes back to the same time when fundamental concepts of neural networks were discovered. In the paper *Optimal Brain Damage* [9], the concept of "saliency" was presented as the effect of removing a weight from the network on the loss or objective function [9]. This technique involves training the model until a reasonable solution, then compute the saliency of the weights to prune, and finally fine-tune the model [9]. The saliency was based on the second derivative of the loss with respect to weights, in other words, the Hessian matrix [9]. Another work that has extended the *Optimal Brain Damage* [9] is the *Optimal Brain Surgeon* [10], which has shown a problem with the calculation of the saliency in the *Optimal Brain Damage* [9]. Another thing to note is that the *Optimal Brain Surgeon* technique [10] does not require fine-tuning [11].

Another approach to pruning which is newer and more fundamental for the other researches to come was presented by Han et al. [12]. In their paper, Han et al. proposed a technique to learn the important connection along with the efficient weights while training process of a model [12]. After an initial training phase, weights that are smaller than a threshold are removed, resulting in a sparser model [12]. To compensate the loss in accuracy, the model is then retrained [12]. This approach is said to be a fundamental approach in the pruning research. It is even said that many papers are slight variations of this approach [3].

In a more recent meta-analysis of the papers on pruning methods, Cheng et al. [1] has found more diverse techniques that were more recent. Their taxonomy [1] of the techniques is as follows:

- Universal/Specific Speed-up
 - Unstructured
 - Structured
 - Semi-structured
- When to prune
 - Pruning before training
 - Pruning during training
 - Pruning after training
 - Run-time pruning
- Pruning criteria
 - Magnitude
 - L_p norm
 - Saliency
 - Loss Change

Original taxonomy includes more sections, here, the relevant ones were chosen.

Among the techniques that explored when to prune, *Pruning during training* (PDT) was explored less due to the more complicated dynamic process compared to others [1]. The papers that were selected by Cheng et al. [1] as representatives of PDT are Network Slimming [13], Sparse Structure Selection (SSS) [14], Sparse Evolutionary Training (SET) [15], Differentiable Sparsity Allocation (DSA) [16], GraNet [17], and FreeTickets [18].

Network Slimming [13] focuses on imposing L1 regularization on the scaling factors in batch normalization (BN)

layers, claiming that pushing the values of BN scaling factors towards zero with L1 regularization allows the identification of insignificant channels in Convolutional Neural Networks (CNNs). In their paper SSS, Huang and Wang [14], approached the pruning as a joint sparse regularized optimization problem by introducing scaling factors and corresponding sparse regularizations. The method includes modified stochastic Accelerated Proximal Gradient (APG) method to jointly optimize the weights of CNNs and scaling factors with sparsity regularizations [14].

The SET [15] method is different from other papers in the sense that the method starts with a sparse network. In their paper Mocanu et al. [15] states that topological sparsity must be pursued starting with the design phase of the neural networks. They claim that a sparse network trained with SET does not have a loss in accuracy but has quadratically fewer parameters than a typical neural network [15].

For the DSA method, Ning et al. says that this method is a differentiable pruning process, which enables the gradients of the loss with respect to the pruning ratios to be calculated [16]. They also claim that the gradient magnitudes align well with the layer-wise sensitivity (effect of layer on the loss), which can act as a measure of the sensitivity [16].

The main difference of GraNet [17] from other methods is that it does not only consider pruning the weights during training, but also introduce the concept of "neurogeneration", which was inspired by how human brain works. The neuro-generation refers to the process of pruning some connections in the network but also adding same amount of connections [17]. The metric that was chosen to both remove and add connections is the magnitudes of the gradients of the weights [17].

Last but not least, *FreeTickets* is different from previous methods in the sense that it is an ensemble method [18]. Along the line of how ensemble methods work, this method independently trains multiple subnetworks from scratch with dynamic sparsity and then averages the predictions of these subnetworks [18].

III. PROBLEM STATEMENT

In the papers that were presented in the Section II, it was observed that there are less methods that utilize a pruning technique that can process the network during the training as most of the techniques first need a model that has converged with a training process. The papers that are utilizing a technique during training often need additional parameters that needs to be calculated during the training or even trained as a parameter along with the existing ones. Another common approach was to utilize a single score, mostly either the magnitudes of the weights of connections, or the gradients of them.

In this paper, the goal is to find a simple method that does not need an additional parameter to be used during training and utilizes both magnitudes and gradients of the weights.

Previous studies, for example [12] and [18], indicate that the magnitudes of weights and gradients are important indicators of weight importance. Inspired by them, it was hypothesized that using both of them at the same time can yield better pre-

diction in terms of understanding which weights are actually important for the loss function.

In a more detailed way, the hypothesis of this paper is that the weights to be pruned must first be decided based on their gradients. As this method is to be applied during training, it can be assumed that optimal weights are not known. However, as the network is updated based on the gradient, we can use the gradient to understand how much update is needed for each weight to get closer to its optimal value. It can be hypothesized that lesser the gradient, less update is needed for a particular weight, thus this weight can be assumed to be closer to its optimal value. Then, for such weights that are assumed to be closer to their optimal values, if the current weight is small in terms of its magnitude, then this weight can be considered as "unimportant". Applying such pruning every few epochs within training also allows the model to adjust the other weights to regain the previous performance.

Further details about how this method was implemented for the experiments are available in Section IV.

IV. METHODOLOGY

This section includes the two algorithms implementing the idea explained in Section III and details about the experiments conducted.

A. Algorithm

The first version for the implementation of the idea that was presented in Section III is Algorithm 1. This algorithm is designed for a simple Multilayer Perceptron (MLP) [6] architecture, further details regarding architecture are available in Section IV-B. It must be noted that this version of the algorithm is unstructured, meaning that it evaluates each of the weights of the connections separately. In Algorithm 1, *Perc* refers to finding the percentile. There are two important things to note about this algorithm. First one is that, threshold for gradients and weights are calculated independent from each other, in order not to miss the overall state of a layer. Each threshold was also calculated per layer, as distributions of gradients can change during backpropagation, from layer to layer. The second thing to note is that pruning refers to assigning 0 to a weight, but this does not strictly stop the weight from being updated further. This was inspired by [17]. Instead of adding new connections, allow the ones pruned to be updated if necessary.

After the evaluation of preliminary results using Algorithm 1, it was decided to extend the experiments with a more common architecture type, such as convolutional neural networks (CNN) [4]. To adapt the idea presented in Section III, Algorithm 2 was designed. The most significant difference between Algorithms 1 and 2 is that the latter implements a structured pruning. This means that it does not consider each weight in each filter of convolutional layers, but it rather evaluates a filter as a whole. This design choice is due to the belief that a filter is a more meaningful unit compared to a single weight within a filter. Similar to Algorithm 1, Algorithm 2 also goes through each convolutional layer separately and decides the thresholds for each layer individually. Because it

Algorithm 1 Two-Stage Unstructured Pruning

Require: Model M with linear layers, percentile p

```

1: for each linear layer  $L$  in  $M$  do
2:   Extract weight matrix  $W$  and gradient matrix  $G$  from  $L$ 
3:   if gradient  $G$  is not None then
4:     Compute gradient threshold  $\tau_g \leftarrow \text{Perc}(|G|, p)$ 
5:     Compute low-gradient mask  $M_g \leftarrow |G| < \tau_g$ 
6:     Compute weight threshold  $\tau_w \leftarrow \text{Perc}(|W|, p)$ 
7:     Compute prune mask  $M_p \leftarrow (|W| < \tau_w) \wedge M_g$ 
8:     Prune weights:  $W[M_p] \leftarrow 0$ 
9:   end if
10: end for
```

is structured, mean absolute weights and gradients were used for each filter to decide the thresholds using percentile value. Again similar to Algorithm 1, if a filter is decided to be pruned, it is not completely removed but rather filled with zeros, as a form of soft pruning.

Algorithm 2 Two-Stage Structured Filter Pruning

Require: Model M , Pruning percentile p

```

1: for all convolutional layers  $L$  in  $M$  do
2:    $W \leftarrow$  weights of  $L$ 
3:    $G \leftarrow$  gradients of  $L$ 
4:   if  $G$  is None then
5:     continue
6:   end if
7:   Compute mean absolute weights:  $\bar{W}_i = \frac{1}{|W_i|} \sum |W_i|$ 
8:   Compute mean absolute gradients:  $\bar{G}_i = \frac{1}{|G_i|} \sum |G_i|$ 
9:    $w_{thresh} \leftarrow p^{\text{th}}$  percentile of  $\bar{W}$ 
10:   $g_{thresh} \leftarrow p^{\text{th}}$  percentile of  $\bar{G}$ 
11:  for all filters  $i$  in  $L$  do
12:    if  $\bar{W}_i < w_{thresh}$  and  $\bar{G}_i < g_{thresh}$  then
13:      Zero out  $W_i$  (soft prune)
14:    end if
15:  end for
16: end for
```

Both versions of algorithms are added to the traditional training loop of a related deep learning architecture right after one epoch pass. It can be applied every few epochs or after each epoch. Further details are available in IV-B.

B. Experiment Design

This section explains the experiment design for the preliminary results obtained with MLP and the new experiments done after that using CNN architectures.

To see that whether the method is working or not and see preliminary results, a simple experiment was designed. The dataset used for training and testing for this simple experiment is MNIST [4], and the model is a simple Multilayer Perceptron (MLP) [6]. 20% of the training set was used as the validation set. The MLP includes 4 hidden layers, 128 neurons each. ReLU was chosen as layer activation function. Cross-entropy loss was utilized as the objective function, as

MNIST dataset consists of handwritten digits and these digits are labeled with which digit they actually are. Images were normalized before passing them to the network. Learning rate that was used for all of the experiments is 0.001 and all networks were trained for 20 epochs, with a batch size of 64 images. ADAM [19] was chosen as the optimizer in all experiments. Implementations were done using Pytorch [20]. Experiments were run on Google Colab's [21] GPU backend.

Two sets of experiments were designed for preliminary results: Without L1 regularization and with L1 regularization. For each set, experiments were done for the number of epochs that the pruning is done after and the percentile. Values chosen for number of epochs as interval, in other words, after how many epochs we should apply the pruning, is 5 and 1. Percentiles that were tested are 10%, 30%, 50%, 70% and 90%.

To see how well the method works for a CNN architecture, experiments were repeated with ResNet18 [7] architecture and CIFAR-10 [5] dataset, inspired by the survey paper [3]. 20% of the training set was used as the validation set. Cross-entropy loss was utilized as the objective function, as CIFAR-10 is a dataset for image classification with 10 classes [5]. Similar to first set of experiments, images were normalized before passing them to the network. Learning rate that was used for all of the experiments is 0.001 and all networks were trained for 30 epochs, with a batch size of 128 images. ADAM [19] was chosen as the optimizer in all experiments. Implementations were done using Pytorch [20]. Experiments were run on Google Colab's [21] A100 backend.

Rest of the design for experiments with ResNet18 [7] is same with first set of experiments: with and without L1 regularization, number of epochs as interval is 5 and 1, 10%, 30%, 50%, 70% and 90% as percentile values.

By looking at the results of ResNet18 [7], it was decided to repeat a similar set of experiments with VGG16 architecture [8] for comparison. Almost all details of experiments for VGG16 [8] are same with the experiments done with ResNet18 [7], except that training with regularization was not included for VGG16 [8] due to the limited credits remaining for A100 usage on Google Colab [21], and the learning rate is 0.0005.

V. NUMERICAL RESULTS

In this section, results of the experiments described in Section IV-B are presented. Sparsity refers to the ratio of individual weights that are equal to 0 to the total number of weights in the model.

A. MLP & MNIST

Tables I and II refer to the preliminary results with and without regularization, respectively.

The loss and accuracy graphs presented for preliminary experiments, Figure 1 and Figure 2, are examples of how losses and accuracies change over the epochs. For these examples, 1 refers to the experiment with regularization, interval=5 and percentile=90. Figure 2 refers to the experiment without regularization, interval=1 and percentile=90.

TABLE I
MLP & MNIST: PERFORMANCE COMPARISON WITH DIFFERENT SPARSITY LEVELS WITH REGULARIZATION

Model	Sparsity	Test Accuracy
Original	0%	97.63%
Interval = 5, Percentile = 10	0.59%	97.87%
Interval = 1, Percentile = 10	1.07%	97.45%
Interval = 5, Percentile = 30	5.93%	97.44%
Interval = 1, Percentile = 30	5.45%	97.52%
Interval = 5, Percentile = 50	20.15%	97.97%
Interval = 1, Percentile = 50	20.90%	97.86%
Interval = 5, Percentile = 70	52.91%	97.64%
Interval = 1, Percentile = 70	52.97%	97.88%
Interval = 5, Percentile = 90	82.11%	97.22%
Interval = 1, Percentile = 90	82.23%	97.90%

TABLE II
MLP & MNIST: PERFORMANCE COMPARISON WITH DIFFERENT SPARSITY LEVELS WITHOUT REGULARIZATION

Model	Sparsity	Test Accuracy
Original	0%	97.76%
Interval = 5, Percentile = 10	1.10%	97.87%
Interval = 1, Percentile = 10	1.30%	97.80%
Interval = 5, Percentile = 30	8.13%	97.14%
Interval = 1, Percentile = 30	9.37%	97.63%
Interval = 5, Percentile = 50	25.66%	97.67%
Interval = 1, Percentile = 50	26.98%	97.61%
Interval = 5, Percentile = 70	49.60%	97.57%
Interval = 1, Percentile = 70	50.24%	97.76%
Interval = 5, Percentile = 90	81.51%	94.07%
Interval = 1, Percentile = 90	81.76%	97.99%

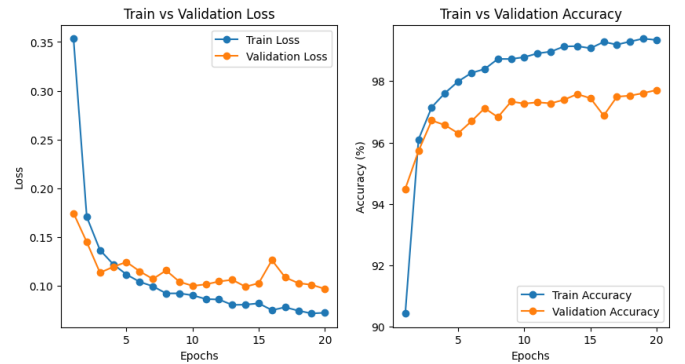


Fig. 1. MLP & MNIST: Example loss and accuracy graphs with respect to epochs for Interval=5, Percentile=50 with regularization.

In Figures 3 and 4, an overall representation of how the percentile chosen affects the final sparsity of the pruned MLP and the final test accuracy of the pruned MLP, respectively.

B. ResNet18 & CIFAR-10

Tables III and IV refer to the ResNet18 and CIFAR-10 results, with and without regularization, respectively.

The loss and accuracy graphs presented for the experiments with ResNet18 and CIFAR-10, Figures 5, 6, 7 and 8, are examples of how losses and accuracies change over the epochs.

For these examples, Figure 5 refers to the experiment with regularization, interval=1 and percentile=50. Figure 6 refers to the experiment with regularization, interval=5 and

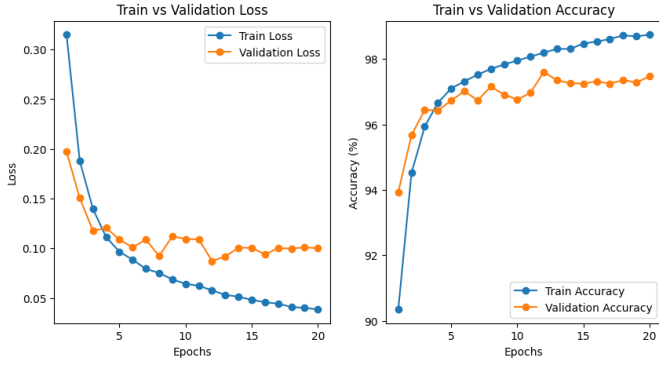


Fig. 2. MLP & MNIST: Example loss and accuracy graphs with respect to epochs for Interval=1, Percentile=90 without regularization.

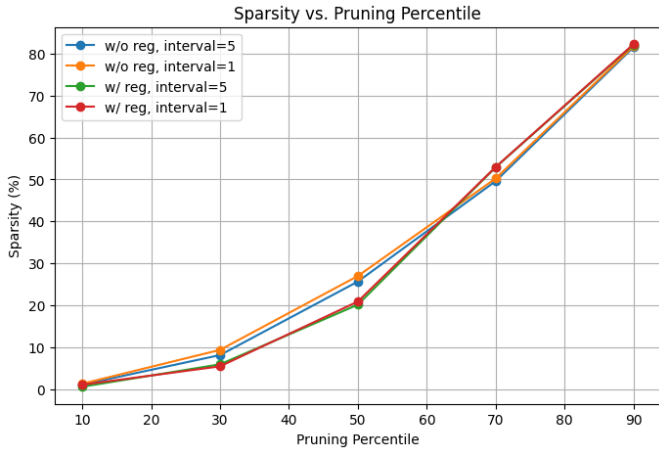


Fig. 3. MLP & MNIST: Sparsity vs Pruning Percentile.

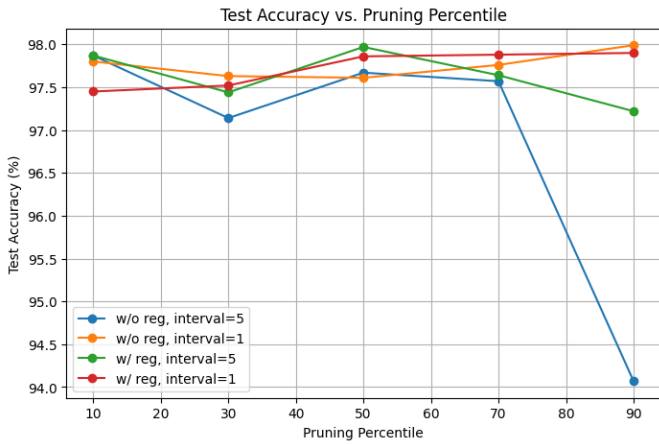


Fig. 4. MLP & MNIST: Accuracy vs Pruning Percentile.

percentile=70. Figure 7 refers to the experiment without regularization, interval=1 and percentile=50. Figure 8 refers to the experiment without regularization, interval=5 and percentile=70.

In Figures 9 and 10, an overall representation of how the percentile chosen affects the final sparsity of the pruned ResNet18 and the final test accuracy of the pruned ResNet18 on CIFAR-10, respectively. The red dashed line in Figure 10

TABLE III
RESNET18 & CIFAR-10: PERFORMANCE COMPARISON WITH DIFFERENT SPARSITY LEVELS WITH REGULARIZATION

Model	Sparsity	Test Accuracy
Original	0%	82.52%
Interval = 5, Percentile = 10	3.68%	81.41%
Interval = 1, Percentile = 10	4.71%	79.76%
Interval = 5, Percentile = 30	20.66%	80.52%
Interval = 1, Percentile = 30	19.31%	83.18%
Interval = 5, Percentile = 50	35.12%	77.13%
Interval = 1, Percentile = 50	39.52%	80.21%
Interval = 5, Percentile = 70	63.13%	61.57%
Interval = 1, Percentile = 70	62.45%	75.77%
Interval = 5, Percentile = 90	83.39%	12.11%
Interval = 1, Percentile = 90	80.69%	10.15%

TABLE IV
RESNET18 & CIFAR-10: PERFORMANCE COMPARISON WITH DIFFERENT SPARSITY LEVELS WITHOUT REGULARIZATION

Model	Sparsity	Test Accuracy
Original	0%	82.88%
Interval = 5, Percentile = 10	3.90%	81.58%
Interval = 1, Percentile = 10	4.38%	80.19%
Interval = 5, Percentile = 30	17.18%	77.22%
Interval = 1, Percentile = 30	15.79%	83.14%
Interval = 5, Percentile = 50	35.32%	73.48%
Interval = 1, Percentile = 50	36.44%	81.91%
Interval = 5, Percentile = 70	57.86%	16.50%
Interval = 1, Percentile = 70	57.61%	80.15%
Interval = 5, Percentile = 90	85.08%	10.09%
Interval = 1, Percentile = 90	82.10%	10.04%

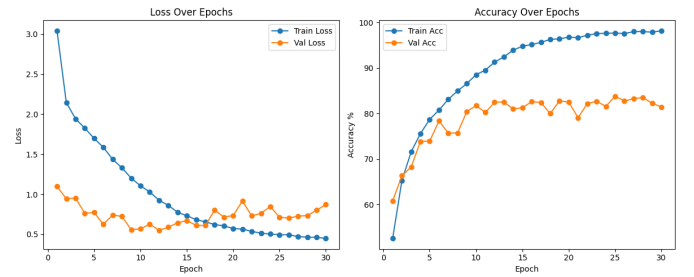


Fig. 5. ResNet18 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=1, Percentile=50 with regularization.

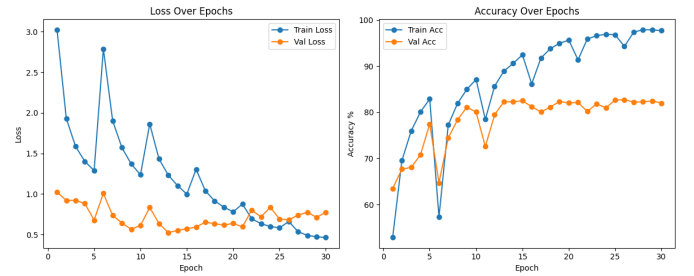


Fig. 6. ResNet18 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=5, Percentile=70 with regularization.

refers to the accuracy achieved without any pruning.

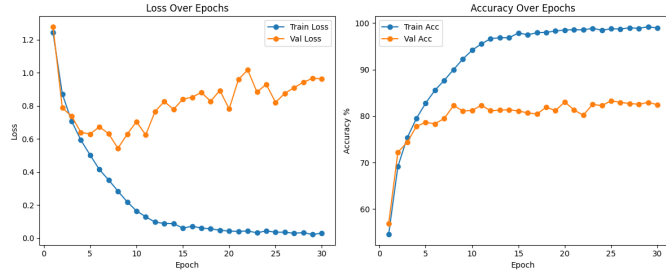


Fig. 7. ResNet18 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=1, Percentile=50 without regularization.

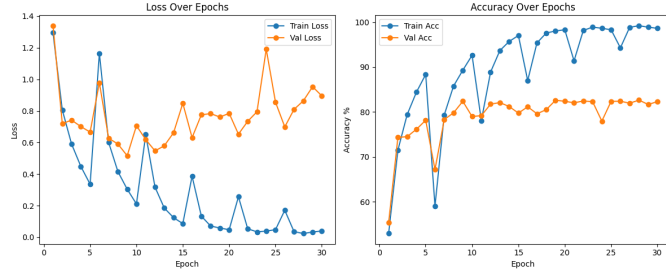


Fig. 8. ResNet18 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=5, Percentile=70 without regularization.

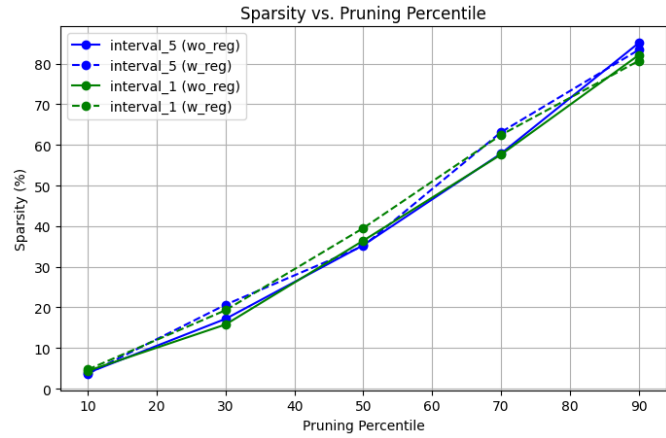


Fig. 9. ResNet18 & CIFAR-10: Sparsity vs Pruning Percentile.

C. VGG16 vs. ResNet18

Table V refers to the VGG16 and CIFAR-10 results, without regularization.

The loss and accuracy graphs presented for the experiments with VGG16 and CIFAR-10, Figures 11, 12, and 13, are examples of how losses and accuracies change over the epochs.

In Figure 14, an overall representation of how the chosen percentile affects the final sparsity of both pruned ResNet18 and pruned VGG16, without regularization. The function plotted in blue in this figure corresponds to the squares of percentile values. The baseline values on the graphs refer to the accuracies achieved without any pruning.

In Figure 15, the final test accuracies of both pruned ResNet18 and pruned VGG16 on CIFAR-10 dataset are presented.

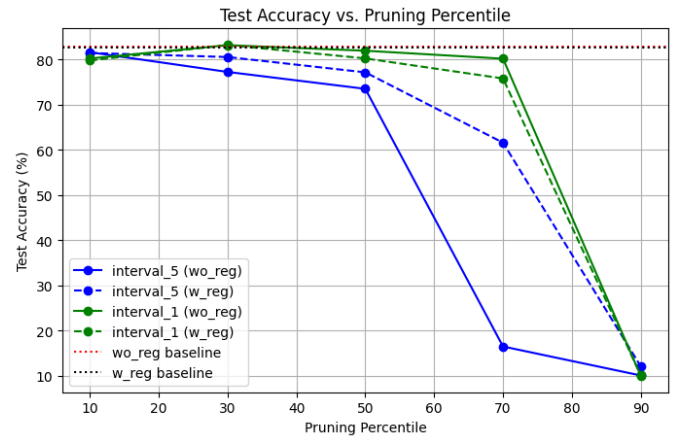


Fig. 10. ResNet18 & CIFAR-10: Accuracy vs Pruning Percentile.

TABLE V
VGG16 & CIFAR-10: PERFORMANCE COMPARISON WITH DIFFERENT SPARSITY LEVELS WITHOUT REGULARIZATION

Model	Sparsity	Test Accuracy
Original	0%	77.50%
Interval = 5, Percentile = 10	0.96%	77.01%
Interval = 1, Percentile = 10	1.09%	78.64%
Interval = 5, Percentile = 30	5.53%	77.90%
Interval = 1, Percentile = 30	6.72%	77.79%
Interval = 5, Percentile = 50	16.78%	76.37%
Interval = 1, Percentile = 50	16.98%	76.92%
Interval = 5, Percentile = 70	46.17%	75.56%
Interval = 1, Percentile = 70	40.33%	73.59%
Interval = 5, Percentile = 90	60.21%	26.90%
Interval = 1, Percentile = 90	61.02%	66.90%

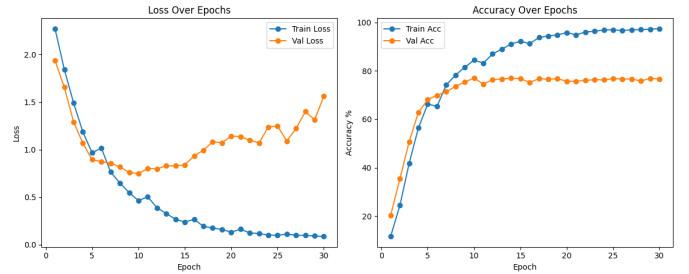


Fig. 11. VGG16 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=5, Percentile=70 without regularization.

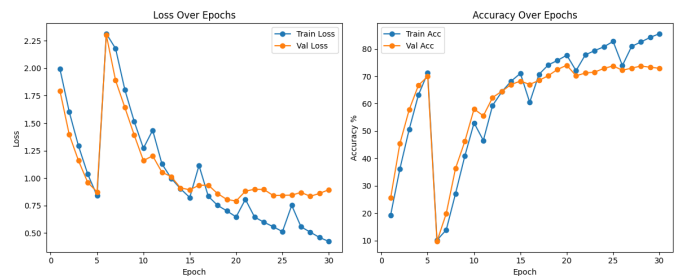


Fig. 12. VGG16 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=5, Percentile=90 without regularization.

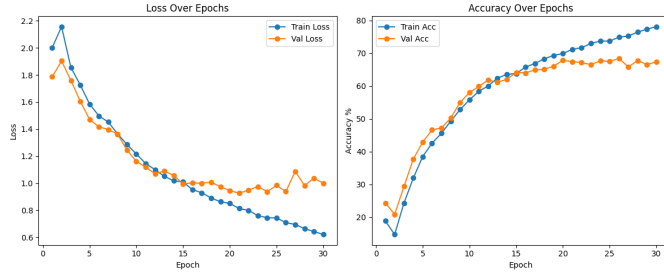


Fig. 13. VGG16 & CIFAR-10: Example loss and accuracy graphs with respect to epochs for Interval=1, Percentile=90 without regularization.

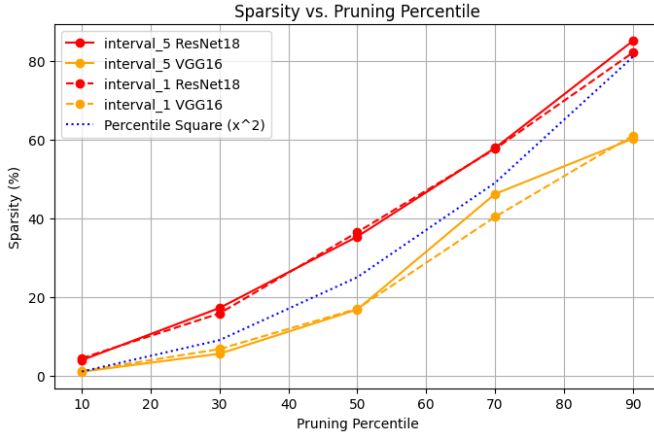


Fig. 14. ResNet18 vs VGG16: Sparsity vs Pruning Percentile.

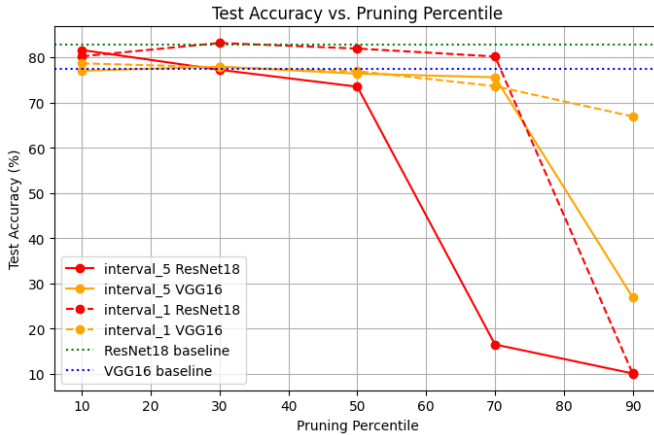


Fig. 15. ResNet18 vs VGG16: Accuracy vs Pruning Percentile.

VI. DISCUSSION

A. Evaluating Results

The main result that can be observed from the preliminary experiment results presented in Tables I and II is that method manages to induce sparsity in the MLP while keeping the test accuracies almost same without significant drops. In some cases, the pruned model performs slightly better than the original model. This indicates that the hypothesis is supported by the preliminary experiments.

Another observation is that how sparsity percentage is related to percentile. It can be seen that sparsity ratio is close

to square of percentile used during the pruning. This effect is especially visible in Table II.

Not all loss and accuracy graphs have been presented in Section V as they all look almost same for the preliminary experiments. This indicates that the method does not disturb the learning process deeply. Convergence on both the training and validation losses is still visible. One note is that when interval=1, validation loss and accuracy seem to converge more smoothly in general.

The last comment on the training process for preliminary experiments is that especially with the experiments conducted without regularization, the sparsity ratio achieved after the first pruning is close to the ratio achieved at the very end of training. One can claim that the method is successful in terms of finding the unimportant weights even after few epochs of training.

However, due to the simplicity of classification task with MNIST and capability of MLP being enough for it, one can argue that the good results are due to this simplicity in experiment design. That is why the second set of experiments with ResNet18 and CIFAR-10 was conducted.

The results of experiments with ResNet18 and CIFAR-10 tell a more interesting story. First of all, Tables III and IV show us that most of the time, the sparsity ratio is higher than square of the percentile used. It is even slightly higher in Table III, in other words, with regularization. Figure 9 shows this observation more clearly.

However, even a more interesting observation was done after looking at the accuracy results. Figure 10 shows us that up until 50th percentile, training with interval 5 was around acceptable accuracy results, despite the drop in it, but with 70th percentile, accuracy level is beyond acceptable. We can see that regularization helped to keep the accuracy higher, but it is still significantly far from baseline accuracy. On the other hand, we can see that training results with interval 1 seem to be much more robust against increasing percentile & decreasing accuracy trade-off. Though, without regularization seems to be better with interval 1. Regardless of the other hyperparameters, at 90th percentile, performances become useless. In comparison to preliminary results, we can see that drop in accuracy is much clearer with a large architecture like ResNet18, and yet with correct hyperparameters such as percentile, interval value and regularization, 57.86% sparsity can be achieved while dropping the accuracy only for 3%.

A possible explanation for why interval 1 works better for interval 5 can be due to the fact that interval 1, in other words pruning after each epoch, can be good at ensuring that sparsity is consistent over all epochs and training of the weights were forced to adapt to it, causing a similar effect like regularization. However, further tests are needed to prove this claim.

Further observation on training loss and accuracy graphs show that training is disturbed more compared to preliminary experiments. First of all, Figure 7 shows that even if the training loss is converging fine, validation loss did not always converged smoothly, on the contrary, it slightly increased over epochs. It did not get worse that what it was in the first epoch, but it is still worth taking a note that validation loss is instable. In Figures 6 and 8, we can see the effect of interval

5 much clearer. The spikes correspond to the epochs where pruning was done. Another observation is that convergence was delayed with higher percentile values. Figure 5 does not show a clear convergence, even with training loss. It seems like training should be continuing after last epoch. Despite all of this disturbance in loss, validation accuracies seem to be disturbed much less than losses and convergence can be seen, but it would be a good question to ask what would be the case for other performance metrics.

Another important note about validation and test accuracies is that they are not close for higher percentile values. For example, in Figure 8 we can see that final validation accuracy is around 60%. However, in Table IV, we can see that this hyperparameter configuration has led to a test accuracy of 16.50%. There is something to remember before attempting to explain it, training ends with a final pruning and no further finetuning after that. This was the case for all experiments in this paper, so there are cases that the final model performed good regardless of this fact. However, because the difference is much clearer with higher percentile, it can be argued that if these models were trained for one more epoch, they could have regained the performance. In the context of these experiments, it still would not have changed the results because the accuracies would still be low compared to baseline, but it is something worth to note. Though, it must also be noted that training was completely unstable in every sense for 90th percentile, regardless of the hyperparameters.

As a small ablation study to see whether the percentile value causing significant drop in accuracy is related to model architecture or not, a similar set of experiments were repeated with VGG16. Details of the experiments can be found in Section IV-B, note that all experiments for this part are without regularization. In Figure 14, we can see the comparison between ResNet18 and VGG16 in terms of induced sparsity percentages. The blue line represents the square of the percentiles. We can see that the method have induced more sparsity for ResNet18 compared to VGG16. In Figure 15, we can see that VGG16 is much more robust compared to ResNet18 to the pruning especially with higher percentile values. Only part that interval value causes a difference is at 90th percentile. As a candidate for best configuration for sparsity & drop in accuracy trade-off, we can look at interval 5, percentile 70 configuration with VGG16, where sparsity induced is 46.17% and the drop in accuracy is only around 2%. In other words, different architectures do respond differently to the proposed pruning method.

For the training metrics, up until 70th percentile, training seems to be not disturbed and quite close to the baseline training. With 70th percentile, validation loss has slightly increased over the epochs, as it can be seen in Figure 11. Only with 90th percentile we see much clearer disturbance in training, for example in Figure 12 we can see the spikes and in Figure 13 we can see that convergence of training loss has slowed down greatly.

B. Future Work

Even though results have shown that the proposed pruning method is successful at inducing sparsity without a significant

drop in accuracy using the right hyperparameters, there are questions that need to be looked into more deeply.

One of them is how other performance metrics like F1 score would look like during training with pruning, or how they would have been affected by it.

Another one is more fundamental, how this pruning method would affect other state-of-the-art CNN models. Even for the architecture types used in this project, ResNet and VGG, there are different versions with different number of parameters. Experiments can be extended to cover more models and datasets.

For this method to be comparable with other existing methods, there is another issue to tackle. The sparsity ratio that is aimed to be achieved with pruning can be given as a parameter to other pruning methods, whereas the proposed method does not take such parameter. The final sparsity ratio can be guessed, but results showed that there is still some randomness involved. A way to ensure a specific sparsity level can be searched for the proposed method.

Finally, adaptation of this method to other common deep learning architectures like Transformers [22] would help with usefulness of the method.

VII. CONCLUSION

In this paper, a simple approach to pruning during training was introduced. The main novelty of this method compared to previous studies is to combine the gradients and the weights of the parameters in a model to decide the pruning, during the training, without the need of additional parameters. This method does not wait for a model convergence, rather it utilizes gradients as a predictor for whether the weight is close to its optimal value while still optimizing the weights.

Preliminary results show that method manages to induce sparsity in the model without a significant drop in the accuracy. Further experiments with two commonly known CNN architectures, ResNet18 [7] and VGG16 [8] also showed that method can induce sparsity without a significant drop in accuracy, if correct hyperparameters are used. The results have also shown that different model architectures are affected differently in terms of sparsity ratio, the drop in accuracy and the stability of the training.

With further experiments and exploration, it can be seen that how useful this method can be in practical sense.

APPENDIX A

COMMENTS ON RESEARCH PROPOSAL

Hi Pelinsu, This is an interesting project that holds great promise. I believe the results of this research could be quite significant for practical applications where a large deep learning network might not be suitable for implementing on small devices and pruning might be a suitable approach for this. Having said that, it remains to be seen how sensitive such pruned deep learning ANN would be to distribution shift or domain adaptation issues. One can clearly see that there is major tradeoff here between hardware and software implementation efficiency versus the accuracy of the predictions of such a pruned deep network. I am curious to see if there is

some kind of a "sweet spot" here which is a good engineering tradeoff. Using explainable AI would certainly be another interesting direction to pursue for pruning. I wonder if there are fundamental limits that one might come up with as a result of your investigation. For example, for given system parameters, the best one can do is 25% pruning.....I think this, as you correctly observed, is a very timely research topic that is quite rich in nature and could help with many roadblocks in engineering design of deep artificial neural networks (ANN)

APPENDIX B

COMMENTS ON MIDTERM PROGRESS REPORT

Dear Pelinsu:

I enjoyed reading your report as it is an interesting research direction that has practical significance.

Your preliminary results indeed look very promising. I look forward to seeing more comprehensive and conclusive results in your final report.

Please also consider the earlier comments and suggestions I provided to you when you submitted your proposal (you can see them at the end of your report).

One other interesting question that you might need to investigate is the price paid for this pruning approach. As you know, "there is no free lunch" in engineering or AI. One would expect that the prediction accuracy of your Deep Learning approach would suffer due to pruning. Table 1 and 2, however, do not verify this intuition. It's not clear to me why this is the case. You might want to design more extensive test cases to see whether this is a real trend or an artifact due to the simplicity of the toy example you chose.

Best wishes,
Ozan

APPENDIX C

IS IT PUBLISHABLE?

I believe that the report is not publishable as is. Main reason behind the way I think is that I believe the literature review could have been stronger and it also needs better comparison with existing works to be acceptable in a conference. To prove that this method is really useful, much more extensive experiments are needed as the number of variables regarding the method is quite large. Last but not least, it would have been better to see the effects of the method on training resources, but because I have done the experiments on a remote server, I do not have data regarding this aspect.

REFERENCES

- [1] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [3] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Gutttag, "What is the state of neural network pruning?" *Proceedings of machine learning and systems*, vol. 2, pp. 129–146, 2020.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009, technical Report.
- [6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [9] S. Solla, Y. Le Cun, and J. Denker, "Optimal brain damage," *Advances in neural information processing systems*, vol. 2, pp. 598–605, 1990.
- [10] B. Hassibi and D. Stork, "Second order derivatives for network pruning: Optimal brain surgeon," *Advances in neural information processing systems*, vol. 5, 1992.
- [11] T. Choudhary, V. Mishra, A. Goswami, and J. Sarangapani, "A comprehensive survey on model compression and acceleration," *Artificial Intelligence Review*, vol. 53, pp. 5113–5155, 2020.
- [12] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, vol. 28, 2015.
- [13] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2736–2744.
- [14] Z. Huang and N. Wang, "Data-driven sparse structure selection for deep neural networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 304–320.
- [15] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, "Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science," *Nature communications*, vol. 9, no. 1, p. 2383, 2018.
- [16] X. Ning, T. Zhao, W. Li, P. Lei, Y. Wang, and H. Yang, "Dsa: More efficient budgeted pruning via differentiable sparsity allocation," in *European Conference on Computer Vision*. Springer, 2020, pp. 592–607.
- [17] S. Liu, T. Chen, X. Chen, Z. Atashgahi, L. Yin, H. Kou, L. Shen, M. Pechenizkiy, Z. Wang, and D. C. Mocanu, "Sparse training via boosting pruning plasticity with neuroregeneration," *Advances in Neural Information Processing Systems*, vol. 34, pp. 9908–9922, 2021.
- [18] S. Liu, T. Chen, Z. Atashgahi, X. Chen, G. Sokar, E. Mocanu, M. Pechenizkiy, Z. Wang, and D. C. Mocanu, "Deep ensembling with no overhead for either training or testing: The all-round blessings of dynamic sparsity," *arXiv preprint arXiv:2106.14568*, 2021.
- [19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [21] E. Bisong, "Google colab," <https://colab.research.google.com>, 2019, accessed: 2025-03-14.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>